

Production Agentic Platform for AI-Assisted Engineering Workflows

A plain-English explanation of the project, every component, the end-to-end workflow, and the architecture

The project in one sentence

A platform that lets AI agents do real engineering work — reading code, running builds, opening pull requests — safely, by forcing every action the agent wants to take through a permission check, a risk score, and (when the action is risky) a human approval, while continuously measuring whether the answers are actually correct.

Headline	What it means in plain terms
Serves 1,000+ daily users	About a thousand engineers use it every working day, so it has to behave like a real product, not a demo.
Sub-second latency	The user starts seeing a response in under one second, so it feels instant rather than laggy.
Production accuracy lifted 35%	After the improvements, the system produced a correct, usable result 35% more often than the version it replaced.
Scoped tool permissions	Each user and each agent can only touch the specific tools and resources they are entitled to touch.
Approval gates	Anything that changes the real world waits for a person to click Approve.
Human-in-the-loop escalation	When the agent is unsure or keeps failing, the task is handed to a human expert instead of guessing.
Offline and online evaluation	Quality is tested on a fixed exam before release, and measured again on live traffic afterwards.

Contents

1. What this project is, in simple terms
2. The problem it solves (before and after)
3. Plain-English glossary of every term in the summary
4. The architecture diagram
5. Walking through the architecture, layer by layer
6. Component-by-component deep dive
7. The end-to-end workflow
8. A fully worked example, step by step
9. How evaluation works (offline and online)
10. How the headline numbers were achieved
11. Technology stack summary
12. Design decisions and trade-offs
13. Questions you may be asked, with answers

1. What this project is, in simple terms

Think of an ordinary AI chatbot. You ask it something, it writes an answer back. It cannot do anything — it can only talk.

This project is the next step up. It is a platform where the AI is given hands. It can open your code, read a failing test, look at a build log, search internal documents, comment on a ticket, and raise a pull request. Because it can now actually change things, the interesting engineering problem stops being "can the AI write a good answer" and becomes "how do we let it act without it ever doing something damaging."

So the platform is really two things bolted together:

- **A brain that plans and acts.** It takes one messy human goal ("figure out why checkout is failing and fix it") and turns it into a sequence of small, concrete steps. After each step it looks at what came back and decides what to do next.
- **A set of brakes and seatbelts.** Before any step touches a real system, the platform checks whether this user is allowed to do that, how dangerous the action is, and whether a human needs to approve it first. Everything is recorded.

On top of both sits a measurement system. Every run is scored, so the team can prove the platform is getting better rather than just feeling that it is. That measurement is what produced the 35% accuracy improvement — it was found and verified by testing, not assumed.

A simple analogy

Imagine hiring a very fast, very well-read junior engineer who has never worked at your company.

The brain (LangGraph) is that engineer's ability to plan work and carry it out step by step.

The tools (MCP) are the keyboard, the terminal and the repository access you hand them.

The governance layer is the badge system: which rooms their badge opens, which actions need a manager's signature, and the log of every door they went through.

The evaluation system is their performance review — a fixed set of tasks you check them on, plus real feedback from the people they help every day.

2. The problem it solves (before and after)

What engineering teams were doing before

- Engineers used AI in a separate chat window, then copied and pasted code back and forth by hand.
- The AI could not see the actual repository, the actual failing test or the actual build log, so its suggestions were generic and often wrong for that codebase.
- Teams that did wire an AI directly into their systems had no safety layer, so a single bad instruction could delete a branch or write to production.
- Nobody could say whether the AI was helping. There was no number, only opinions.

What the platform changes

Problem before	How the platform fixes it
The AI had no access to real context, so answers were generic.	MCP tools let the agent read the actual code, logs, tickets and internal documents before answering.
Copy-paste between chat and IDE wasted time.	The agent performs the steps itself and returns a finished pull request link.
Direct system access was dangerous.	Scoped permissions plus approval gates mean nothing risky happens without an entitled user and a human click.
No accountability when something went wrong.	Every tool call, argument and decision is written to an immutable audit log that can be replayed.
No way to know whether quality improved.	Offline evaluation on a fixed test set, plus online evaluation on live traffic, produce a hard accuracy number.
A confidently wrong answer looked the same as a correct one.	The critic step re-checks the work, and low-confidence cases are escalated to a human expert instead of guessed.

3. Plain-English glossary of every term in the summary

The one-line project summary is dense with jargon. Here is every term in it, translated.

Term	What it actually means
Agentic platform	Software where the AI does not just reply — it decides on actions, carries them out using tools, looks at the result, and keeps going until the job is done.
Python	The programming language the backend is written in. It is the standard language for AI work because all the AI libraries live there.
FastAPI	The Python web framework that receives requests over the internet and sends responses back. It is chosen because it is fast, handles many users at once, and checks incoming data automatically.
LangGraph	A library for building an agent as a flowchart, called a graph. Each box in the flowchart is a step, the arrows are the rules for what happens next, and the whole thing keeps a memory of the run so it can pause and resume.
MCP (Model Context Protocol)	A shared standard for describing tools to an AI. Instead of writing custom glue code for each system, every system is wrapped in a small MCP server that advertises what it can do, and the agent talks to all of them the same way.
Tool calling	The AI does not run code itself. It outputs a structured request such as "call read_file with path=tests/checkout_test.py". The platform runs that function and hands the result back.
Scoped tool permissions	A rulebook stating which user or agent may call which tool, on which resource. A junior engineer may read any repository but only open pull requests on their own team's.
Approval gate	A hard stop before an action that changes something real. The agent pauses, a person reviews exactly what it wants to do, and it only proceeds after Approve is clicked.
Human-in-the-loop escalation	When the agent is not confident, or has failed the same step repeatedly, the task is routed to a human expert with all context attached, rather than the agent guessing.
Offline evaluation	A fixed exam. Several hundred saved tasks with known-correct answers are replayed against a new version before it ships. If the score drops, the release is blocked.
Online evaluation	Measuring quality on real traffic after release — comparing two versions side by side and watching whether users accept the results.
Sub-second latency	Less than one second between the user pressing enter and the first useful output appearing.
Production accuracy	The share of real user tasks where the platform produced a correct and usable result, measured on live traffic rather than in a lab.

4. The architecture diagram

The diagram on this page shows the whole system. Read it top to bottom: a request enters at the top and travels down through each layer. The right-hand column shows the three things that run constantly across every layer rather than at one point in the flow.

The single most important idea in the picture is that the arrow from Layer 3 to Layer 5 does not exist. The agent can never reach a tool directly. Everything has to pass through Layer 4 first.

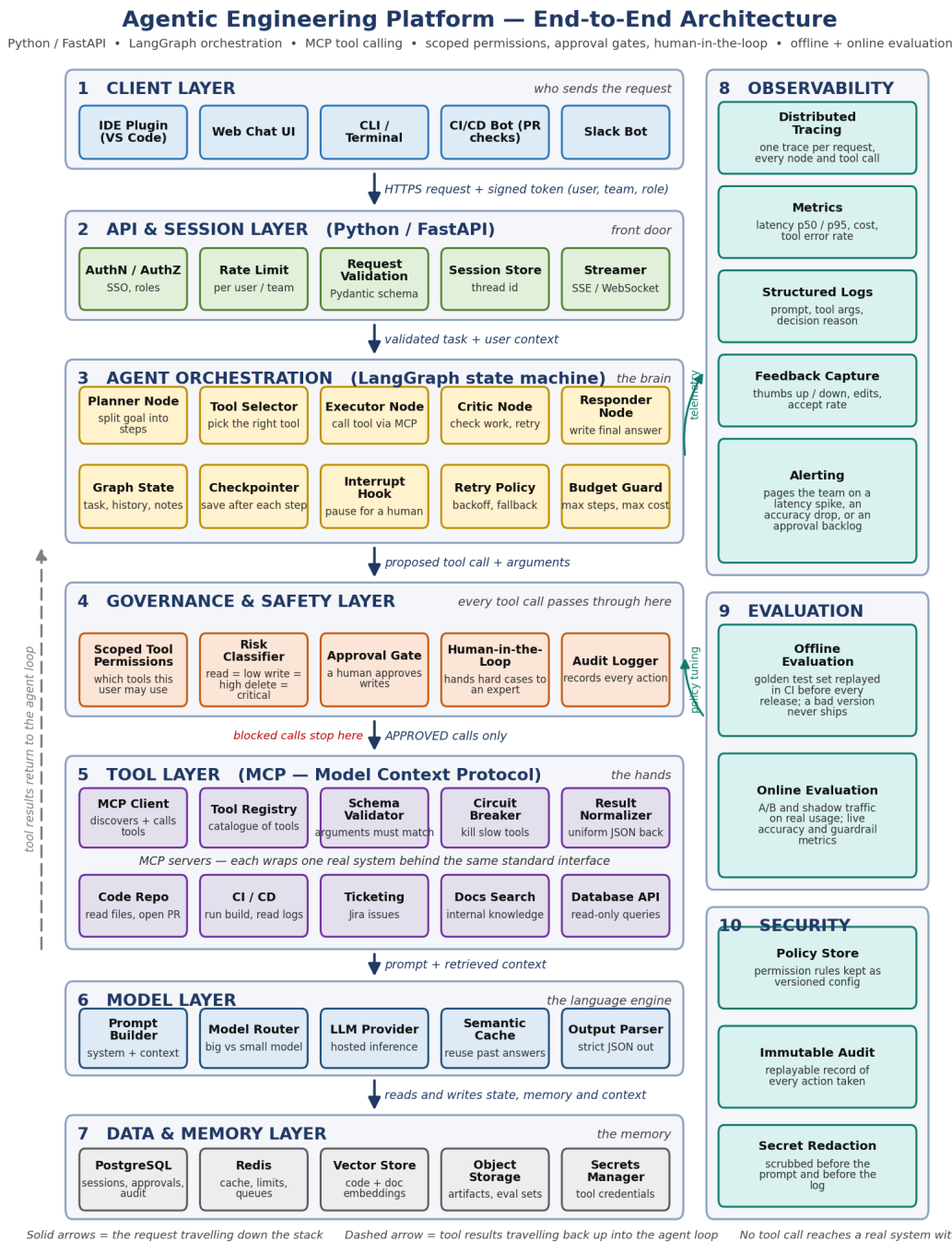


Figure 1 — Full platform architecture. Solid arrows show the request travelling down; the dashed arrow on the left shows tool results returning to the agent loop.

5. Walking through the architecture, layer by layer

This section explains what each numbered block in Figure 1 is for. Section 6 then opens each block up and explains every box inside it.

Layer	Nickname	What it is responsible for
1. Client	Who asks	The places a person or a bot can start a task from: the IDE, a chat window, the terminal, a CI pipeline or Slack.
2. API and session	The front door	Proves who the caller is, rejects malformed or excessive requests, remembers the conversation, and streams output back as it is produced.
3. Orchestration	The brain	The LangGraph state machine that plans the work, chooses one action at a time, checks its own results, and saves its progress after every step.
4. Governance and safety	The gatekeeper	The compulsory checkpoint. Decides whether this user may call this tool, how risky it is, whether a human must approve, and records the decision.
5. Tool layer (MCP)	The hands	The standard interface to every real system — the code repository, CI, ticketing, documentation and internal databases.
6. Model	The language engine	Builds the prompt, picks which model to send it to, calls the model, caches repeat answers, and parses the reply into strict structured data.
7. Data and memory	The memory	Stores sessions, checkpoints, approvals, the audit trail, cached values, document embeddings and credentials.
8. Observability	The instruments	Traces, metrics, logs, user feedback and alerts. Answers "what happened on that run and why".
9. Evaluation	Quality control	Offline testing before release and online testing after it. This is where the accuracy number comes from.
10. Security	The rules and the record	Where permission policy lives, the tamper-proof audit trail, and the redaction of secrets and personal data.

6. Component-by-component deep dive

Each component below is described the same way: what it is, what it does in detail, and why it matters. Where a component is easy to misunderstand, an analogy is included.

Layer 1 — Client layer (who sends the request)

IDE plug-in (VS Code / JetBrains)

In one line: A panel inside the code editor where the engineer types the task.

- It sends the request together with useful local context: which file is open, which lines are selected, which branch is checked out.
- It shows the streamed answer inline and can apply a suggested change directly into the editor.
- It is the highest-value entry point because the engineer never leaves the place they are already working.

Why it matters: Most of the daily usage comes from here. If this is slow or clunky, the whole platform feels slow, regardless of how good the backend is.

Web chat UI

In one line: A browser page for longer conversations and for reviewing what the agent did.

- Used for exploratory questions that are not tied to one file.
- It is also where an approver usually reviews a pending action and clicks Approve or Reject.
- It renders the full trace of a run, so a user can see every tool the agent called and why.

Why it matters: It gives non-editor users, and reviewers, a place to work, and it makes the agent's reasoning visible instead of hidden.

CLI and CI/CD bot

In one line: Automated, non-human entry points.

- The CLI lets engineers script the agent into their own workflows.
- The CI bot fires automatically when a build breaks or a pull request opens, so the agent starts investigating before anyone asks.
- Because there is no human at the keyboard, these callers get stricter permissions and cannot self-approve risky actions.

Why it matters: Automated triggers are where most of the time saving comes from, and also where the safety layer matters most, since nobody is watching in real time.

Layer 2 — API and session layer (Python / FastAPI)

FastAPI application

In one line: The web server that receives every request and returns every response.

- It is asynchronous, which means one server can hold thousands of open connections that are mostly waiting on a model or a tool, without needing thousands of threads.

- This is the single biggest reason the platform can serve over a thousand daily users on modest hardware.
- It is stateless: no user data is kept in the server's memory, so more copies can be started behind a load balancer at any time.

Why it matters: Agent work is dominated by waiting — waiting for the model, waiting for a build log. An async server turns that waiting time into capacity for other users.

Analogy: *A restaurant where one waiter can look after twenty tables, because most of the time each table is eating rather than ordering.*

Authentication and authorisation

In one line: Proving who the caller is, and what role they hold.

- Authentication answers "who are you" — validated through single sign-on and a signed token.
- Authorisation answers "what are you allowed to do" — the user's team and role are extracted and carried with the request for the rest of its life.
- That identity is what Layer 4 later uses to decide whether a tool call is permitted.

Why it matters: Permissions are meaningless without a trustworthy identity. Every later safety check depends on this step being correct.

Rate limiting and quotas

In one line: A cap on how much any one user or team can consume.

- Limits are enforced per user, per team and per tool, using counters held in Redis.
- It protects against a runaway script, an accidental infinite loop, and unbounded model spend.
- When a limit is hit the user gets a clear message rather than a timeout.

Why it matters: One misbehaving caller could otherwise consume the capacity and the budget of everyone else.

Request validation

In one line: Checking the shape of incoming data before anything else runs.

- Every field is declared in a Pydantic schema; anything missing or of the wrong type is rejected immediately with a helpful error.
- This blocks malformed input from ever reaching the agent, where it would cause confusing failures deep in the run.

Why it matters: Failing fast at the door is far cheaper, and far easier to debug, than failing halfway through a multi-step agent run.

Session and thread store

In one line: The platform's short-term conversational memory.

- Each conversation gets a thread identifier; messages, decisions and results are stored against it.
- This is what lets a user say "now do the same for the payments service" and have it understood.

- It is stored in the database, not in server memory, so any server instance can pick up any conversation.

Why it matters: Without it every message would start from zero, and follow-up questions would be impossible.

Streaming (SSE / WebSocket)

In one line: Sending the answer out piece by piece as it is generated.

- Rather than waiting for the complete answer, the first tokens are pushed to the client as soon as they exist.
- Progress events are streamed too, so the user sees "reading the CI log" while it happens.
- This is the mechanism behind the sub-second experience: the number measures time to first visible output, not time to full completion.

Why it matters: A response that begins in 0.4 seconds and finishes in 12 feels fast. One that shows nothing for 8 seconds then appears at once feels broken, even though it is quicker overall.

Analogy: *A kettle with a glass side. The water takes the same time to boil, but you can see it working.*

Layer 3 — Agent orchestration (LangGraph)

This is the brain. LangGraph models the agent as a state machine: a set of nodes (steps), edges (rules for what runs next), and one shared state object that every node reads from and writes to.

The graph state object

In one line: A single shared notebook that travels through the whole run.

- It holds the original goal, the plan, the conversation so far, every tool result, the current step, and running counters for cost and attempts.
- Every node receives the state, returns an updated version of it, and passes it on.
- Because all context lives in one place, any node can see everything that has happened so far.

Why it matters: This is what makes the agent coherent across many steps instead of forgetting what it just did.

Analogy: *A shared case file that each person adds notes to before handing it to the next person.*

Planner node

In one line: Turns one vague goal into an ordered list of small, checkable steps.

- "Find out why checkout is failing and fix it" becomes: read the failing test, fetch the CI log, locate the bug, write a patch, open a pull request.
- The plan is stored in state and can be revised mid-run if a step reveals something unexpected.
- Planning up front is what stops the agent from wandering aimlessly and burning steps.

Why it matters: Small, concrete steps are far more reliable for a model than one enormous instruction, and they are individually checkable.

Tool selector node

In one line: Chooses exactly one tool for the current step, and fills in its arguments.

- It sees only the tools this user is entitled to, which shrinks the choice and improves the hit rate.
- It produces a structured request — the tool name plus its arguments — not free text.
- Selecting one action at a time keeps the run auditable and easy to pause.

Why it matters: Tool choice is one of the most common failure points in agent systems. Constraining the menu to what is permitted measurably improved accuracy.

Executor node

In one line: Hands the chosen call to the governance layer and, if cleared, on to MCP.

- It never calls a tool directly. It submits an intent, which Layer 4 either approves or blocks.
- It writes whatever comes back into the graph state so later nodes can use it.
- Independent calls can be executed in parallel when they do not depend on each other.

Why it matters: Routing every action through one place is what makes the safety guarantee absolute rather than best-effort.

Critic / reflection node

In one line: A second pass that checks the work before it is shown to anyone.

- It re-reads the result against the original goal and asks whether it actually answers it.
- If the patch does not compile, or the answer cites a file that does not exist, it sends the agent back to try a different approach.
- It also produces the confidence signal that decides whether to escalate to a human.

Why it matters: This single node was one of the largest individual contributors to the accuracy improvement, because it catches confidently wrong answers before the user ever sees them.

Analogy: *Proof-reading your own email before hitting send.*

Checkpoint

In one line: Saves the complete state after every single step.

- If the run crashes at step seven, it resumes at step seven rather than starting again.
- It is also what makes pausing possible: the state is written down, the process stops, and hours later it can continue exactly where it left off.
- Every checkpoint is retained, so any run can be replayed for debugging or auditing.

Why it matters: Without checkpointing, a human approval that takes twenty minutes would mean throwing away twenty minutes of work.

Interrupt hook

In one line: The mechanism that lets the graph stop mid-run and wait for a person.

- When an approval is required, the graph raises an interrupt at that exact node.
- The state is checkpointed and the run is parked; no compute is consumed while waiting.

- When the approval arrives, the graph resumes from that node with the human's decision added to state.

Why it matters: This is the technical foundation of both approval gates and human-in-the-loop escalation.

Retry policy and budget guard

In one line: Limits on how hard and how long the agent may try.

- Transient failures such as a timeout are retried with an increasing delay; a repeated failure switches to a fallback approach.
- The budget guard caps the number of steps, the token spend and the wall-clock time for a single task.
- When a budget is exhausted the run stops cleanly and reports what it managed to do, rather than looping forever.

Why it matters: Agents fail in expensive ways. A loop with no limit can spend a large amount of money in minutes and still return nothing.

Layer 4 — Governance and safety layer

This is the layer that makes the difference between a demo and something that can be pointed at a production codebase. Every proposed tool call is stopped here and inspected before it can run.

Scoped tool permissions

In one line: A rulebook mapping identity to allowed tools and allowed resources.

- A rule names the subject (a user, role or agent), the tool, the resource, and the allowed operation — for example: the platform team may write to repositories owned by the platform team only.
- Scoping is not just per tool but per resource, so read access to one repository does not imply access to every repository.
- Rules live in versioned configuration, so a permission change is reviewed and can be rolled back like code.
- Denials are default: if no rule permits an action, it is refused.

Why it matters: This is the difference between giving the agent a master key and giving it a badge that opens three specific doors.

Analogy: *Hotel key cards. Yours opens your room and the gym, not every room on the floor.*

Risk classifier

In one line: Scores how dangerous a proposed action is.

- Reading a file or searching documents is low risk and is allowed to proceed automatically.
- Writing a file, opening a pull request or commenting on a ticket is medium risk and is usually gated.
- Deleting, merging, or anything touching production is high risk and always requires a human.
- The score considers the tool, the target resource, the blast radius, and who is asking.

Why it matters: It keeps the platform fast. If everything needed approval, nobody would use it; if nothing did, it would be unsafe. Risk scoring lets the vast majority of low-risk calls run instantly while genuinely dangerous ones stop.

Approval gate

In one line: The hard stop where a human decides.

- The reviewer sees the exact tool, the exact arguments, the agent's stated reason, and a preview of the effect — for example the actual diff.
- They can approve, reject, or edit the arguments and then approve.
- Approvals are time-limited and single-use, so an old approval cannot be replayed for a different action.
- The decision, the identity of the approver and the timestamp are written to the audit log.

Why it matters: It puts a named human on the hook for every consequential change, which is what makes the platform acceptable to security and compliance reviewers.

Human-in-the-loop escalation

In one line: Routing a task to an expert when the agent should not continue alone.

- Triggered by low critic confidence, by the same step failing repeatedly, by conflicting tool results, or by the task touching a sensitive area.
- The expert receives the full context: the goal, the plan, everything tried, and where it got stuck.
- Their resolution is saved and becomes training material for future evaluation sets.
- This differs from an approval gate: an approval gate asks "may I do this?", escalation says "I do not think I should be the one doing this."

Why it matters: The alternative to escalating is guessing, and a confident wrong answer in a codebase is worse than no answer at all.

Audit logger

In one line: An append-only record of everything that was proposed, decided and done.

- Each entry records who, what tool, what arguments, what the policy decided, who approved, and what the result was.
- Entries cannot be edited or deleted, only added to.
- Combined with checkpoints, any past run can be reconstructed exactly.

Why it matters: When someone asks six months later why a change was made, the answer is a record rather than a recollection.

Layer 5 — Tool layer (MCP)

MCP, the Model Context Protocol, is a shared standard for exposing a tool to an AI system. Before a standard existed, connecting an agent to five systems meant writing five bespoke integrations, each with its own authentication, error handling and data format.

MCP client

In one line: The part of the platform that discovers and calls tools.

- On startup it asks every configured MCP server what tools it offers, what arguments each takes, and what it returns.
- That catalogue is turned into the list of tools the agent is allowed to choose from.
- Because discovery is dynamic, a new tool becomes available by deploying a new server — the platform itself does not need changing.

Why it matters: It turns adding a capability from a code change into a configuration change, which is what let the tool set grow quickly.

Analogy: *A universal remote that asks each new device what buttons it has, instead of needing a new remote per device.*

Tool registry and schema validation

In one line: The catalogue of tools, and the guarantee that arguments are correct.

- Each tool declares its arguments and their types in a schema.
- Every proposed call is validated against that schema before it runs; a malformed call is rejected and the agent is told precisely what was wrong so it can correct itself.
- Descriptions are written carefully, because the quality of a tool description directly affects how often the agent picks the right tool.

Why it matters: Rewriting vague tool descriptions into precise ones was one of the cheapest and most effective accuracy improvements made.

Timeout and circuit breaker

In one line: Protection against slow or broken downstream systems.

- Every tool call has a deadline; if exceeded it is abandoned and reported rather than left hanging.
- If a particular server keeps failing, the circuit breaker trips and calls to it are refused immediately for a cooling-off period.
- The agent is told the tool is unavailable and can plan around it.

Why it matters: Without this, one slow internal service would drag every user's latency up with it.

Analogy: *A fuse box. One faulty appliance trips its own circuit instead of burning the house down.*

Result normaliser

In one line: Turns every tool's output into one consistent shape.

- Different systems return wildly different formats; the normaliser converts them into a uniform structure with a status, the data, and any error.
- Large outputs, such as a fifty-thousand-line build log, are summarised or truncated intelligently so the important part survives.
- Secrets and personal data are stripped before the result is ever placed in a prompt.

Why it matters: It keeps the agent's reasoning simple and stops enormous tool outputs from crowding out the context the model actually needs.

The MCP servers themselves

In one line: One small server per real system, each wrapping it behind the standard interface.

- Code repository server: read files, search the codebase, create branches, open pull requests.
- CI/CD server: trigger builds, read pipeline status, fetch failure logs.
- Ticketing server: read and comment on issues, change status.
- Documentation and search server: retrieve internal knowledge, runbooks and design documents.
- Database and internal API server: deliberately read-only, so the agent can investigate but never mutate data directly.

Why it matters: Each server holds its own credentials, so the agent never sees a token. Compromising the agent does not hand over the keys to the estate.

Layer 6 — Model layer

Prompt builder

In one line: Assembles the exact text sent to the model on every single call.

- It combines the system instructions, the relevant conversation history, the retrieved context, and the list of permitted tools.
- It manages the context budget: when there is too much material, older or less relevant content is summarised or dropped.
- Prompts are versioned, so a change can be evaluated and rolled back like any other code change.

Why it matters: The model can only reason about what it is shown. Most accuracy problems traced back to the wrong context being assembled, not to the model itself.

Model router

In one line: Decides which model handles each call.

- Simple, high-volume calls — classifying intent, picking a tool, summarising a log — go to a small, fast, cheap model.
- Hard reasoning, such as writing a patch, goes to the large model.
- Routing is a major reason both the latency and the cost targets were met at a thousand users a day.

Why it matters: Sending every call to the largest model is slow and expensive; sending every call to the smallest is inaccurate. Routing gets most of the benefit of both.

Semantic cache

In one line: Reuses answers to questions that have effectively been asked before.

- Rather than matching text exactly, it matches meaning, so two differently worded but equivalent questions can share one answer.
- Only deterministic, read-only results are cached; anything that changes state never is.
- Entries expire, so stale answers do not linger after the underlying code changes.

Why it matters: In a large team the same questions recur constantly. Cache hits return in milliseconds and pull the average latency down sharply.

Output parser

In one line: Converts the model's reply into strict, machine-usable data.

- Tool calls must come back as valid structured data matching the expected schema.
- If parsing fails, a repair step asks the model to correct its own output rather than crashing the run.
- This is the boundary where free-form language becomes a reliable instruction the platform can act on.

Why it matters: Everything downstream depends on the model's output being predictable in shape, not just correct in content.

Layer 7 — Data and memory layer

PostgreSQL

In one line: The system of record.

- Holds conversation threads, graph checkpoints, pending and completed approvals, permission decisions and the audit trail.
- Chosen because this data must be transactional and durable — an approval record must never be lost.

Why it matters: It is what allows any server instance to resume any conversation, and what makes the audit trail trustworthy.

Redis

In one line: The fast, short-lived layer.

- Holds rate-limit counters, the semantic cache, task queues and hot session data.
- Sits in front of the database to absorb the high-frequency reads that would otherwise dominate the load.

Why it matters: It removes a large amount of latency from the common path and keeps the database free for work that genuinely needs it.

Vector store

In one line: The searchable memory of the codebase and documentation.

- Code files, design documents and runbooks are converted into numerical embeddings that capture meaning.
- A question is converted the same way, and the closest passages are retrieved and added to the prompt.
- This is retrieval-augmented generation: the model answers using material fetched at question time rather than only what it memorised in training.

Why it matters: It is how the agent gives answers specific to this company's code, rather than generic advice from the public internet.

Analogy: A librarian who fetches the three most relevant pages before you start writing, instead of you reciting from memory.

Object storage and secrets manager

In one line: Bulk artefacts, and credentials.

- Object storage holds large logs, generated diffs and evaluation datasets — things too big for a database row.
- The secrets manager holds tool credentials. They are injected into MCP servers at call time and never enter a prompt, a log or the model's context.

Why it matters: Keeping credentials strictly outside the model's context removes an entire class of leak, including through prompt injection.

Layers 8, 9 and 10 — Observability, evaluation and security

Distributed tracing

In one line: One connected timeline per request.

- Every node, model call and tool call appears as a timed span inside a single trace.
- When a run is slow, the trace shows precisely which span consumed the time.
- When an answer is wrong, the trace shows exactly what the agent saw when it decided.

Why it matters: Agent runs are long and branching. Without tracing, debugging is guesswork; with it, the cause is usually visible in seconds.

Metrics and alerting

In one line: Continuous numbers, and a pager when they move.

- Tracked: time to first token, end-to-end duration, tool error rate, approval wait time, cost per task, escalation rate.
- Latency is watched at the 95th percentile, not the average, because the worst experiences are what users remember.
- Alerts fire on a latency spike, an accuracy drop, a tool failing repeatedly, or an approval queue building up.

Why it matters: Sub-second latency is a promise that has to be defended continuously, not measured once at launch.

Structured logs and feedback capture

In one line: The written record, and the users' verdict.

- Logs record the prompt version, the tools called, the policy decision and the reason, in a queryable format.
- Feedback is captured explicitly through thumbs up and down, and implicitly through whether the user accepted the suggestion, edited it, or discarded it.
- Implicit signals are the more honest measure, because most users never click a rating button.

Why it matters: Feedback is the raw material for evaluation. Whether a suggested change was actually merged is the strongest available signal of real-world correctness.

Policy store, immutable audit and redaction

In one line: The security spine that runs through every layer.

- Permission rules are stored as versioned configuration, reviewed like code, and can be rolled back.
- The audit trail is append-only and replayable.
- Redaction strips secrets, tokens and personal data before content reaches a prompt and again before it reaches a log.

Why it matters: These are the controls that let the platform be approved for use against real production systems rather than a sandbox.

7. The end-to-end workflow

Figure 2 traces one complete task from the moment a user presses enter to the moment they get an answer. The loop in the middle is the important part: the agent does not plan everything and then execute blindly. It takes one action, looks at the result, and decides again.

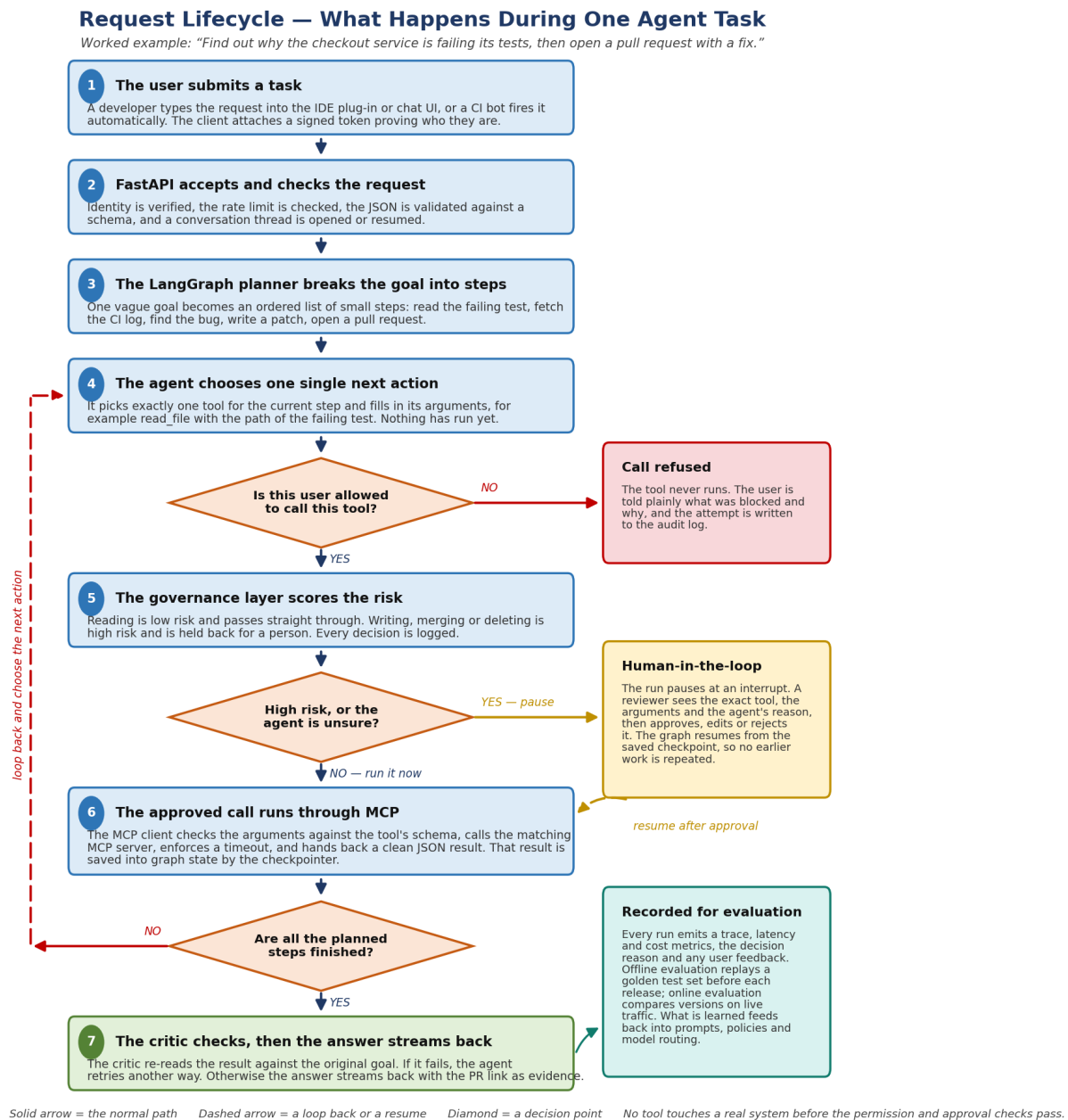


Figure 2 — Request lifecycle. Each numbered box is a stage; each diamond is a decision that can send the run down a different path.

The seven stages in detail

Stage 1 — The user submits a task

- The request arrives from the IDE, the chat UI, the CLI, or automatically from a CI bot when a build breaks.
- A signed token travels with it, carrying the user's identity, team and role.
- Local context is attached where it exists: the open file, the selected lines, the current branch.

Stage 2 — FastAPI accepts and checks the request

- The token is verified and the identity is extracted. An invalid token is rejected here and goes no further.
- Rate limits are checked against Redis counters.
- The request body is validated against a schema; malformed input is rejected with a clear explanation.
- A conversation thread is created or resumed, and a streaming channel is opened so output can flow back immediately.

Stage 3 — The planner breaks the goal into steps

- The LangGraph run begins and the initial state is written: the goal, the user identity, an empty history.
- Relevant context is retrieved from the vector store and attached.
- The planner produces an ordered list of small steps, which is saved into state.
- The plan is not fixed. If a later step reveals something unexpected, the planner revises it.

Stage 4 — The agent chooses one single next action

- For the current step, the tool selector picks exactly one tool and fills in its arguments.
- It only sees tools this user is permitted to use, which narrows the choice and improves the hit rate.
- The output is a structured intent, not free text. Nothing has run yet.

Stage 5 — Governance checks permission and risk

- First question: is this user allowed to call this tool on this resource? If not, the call is refused, the user is told why, and the attempt is logged.
- Second question: how risky is it? Reads pass through; writes and deletions are held.
- If a human is required, the graph raises an interrupt, the state is checkpointed, and the run parks. No compute is burned while waiting.
- The reviewer sees the tool, the arguments, the reason and a preview of the effect, then approves, edits or rejects.
- Every one of these decisions is written to the audit log.

Stage 6 — The approved call runs through MCP

- Arguments are validated against the tool's schema.

- The MCP client calls the matching server, which holds its own credentials, under a strict timeout.
- The result is normalised into a consistent shape, redacted, and summarised if it is very large.
- It is written into graph state and the checkpointer saves progress.
- Control returns to Stage 4 for the next step. This loop repeats until the plan is complete or a budget is exhausted.

Stage 7 — The critic checks, then the answer streams back

- The critic re-reads the accumulated result against the original goal.
- If it does not hold up, the agent retries a different way; if confidence is low, the task is escalated to a human expert.
- If it does hold up, the answer streams back token by token, with the pull request link and every tool call shown as evidence.
- The whole run is recorded: trace, metrics, decision reasons and any feedback, which feeds the evaluation system.

8. A fully worked example, step by step

Task: "The checkout service is failing its tests. Find out why and open a pull request with a fix."

#	What happens	Which components are involved
1	An engineer types the request in the VS Code panel. The plug-in attaches the open branch name and a signed token.	IDE plug-in
2	FastAPI verifies the token, sees the user is on the payments team, confirms they are under their rate limit, validates the body, and opens a thread.	FastAPI, auth, rate limiter, session store
3	Relevant design documents and prior incident notes for the checkout service are retrieved and attached as context.	Vector store, prompt builder
4	The planner produces five steps: read the failing test, fetch the CI log, locate the defect, write a patch, open a pull request.	Planner node
5	The agent chooses read_file on the failing test. This is a read, so it is low risk and passes straight through.	Tool selector, risk classifier
6	The MCP code repository server returns the test file. It is normalised and written into state; a checkpoint is saved.	MCP client, repo server, checkpointer
7	The agent fetches the CI log. It is forty thousand lines, so the normaliser extracts the failing assertion and the surrounding stack trace.	CI/CD server, result normaliser
8	The large model is asked to identify the cause. It finds a null currency code when a discount is applied to an empty basket.	Model router, LLM provider
9	The agent drafts a patch and proposes write_file. This is a write, so the risk classifier marks it medium and the approval gate opens.	Risk classifier, approval gate
10	The graph interrupts and checkpoints. The tech lead sees the exact diff in the web UI and clicks Approve.	Interrupt hook, checkpointer, web UI
11	The run resumes from the saved checkpoint. The patch is written and a pull request is opened, linked to the original ticket.	MCP repo server, ticketing server
12	The critic confirms the patch addresses the failing assertion and that the pull request exists. Confidence is high, so no escalation.	Critic node
13	The answer streams back with the pull request link and the full list of tool calls as evidence.	Streamer, IDE plug-in
14	The trace, latency, cost and the approver's decision are recorded. Days later, the fact that the pull request was merged is captured as a positive outcome signal.	Tracing, metrics, feedback capture

What to notice about this run

The agent never held a credential. Each MCP server used its own.

Eleven of the twelve tool calls were reads and ran with no human involvement at all. Only the one write paused.

The twenty minutes spent waiting for approval cost nothing, because the run was checkpointed and parked rather than kept alive.

The outcome signal — whether the pull request was actually merged — is the honest measure of whether the platform did a good job.

9. How evaluation works (offline and online)

An agent platform can regress in ways ordinary software cannot. A small prompt change or a model upgrade can quietly make results worse while every unit test still passes. Evaluation is the answer to that problem, and it is where the 35% accuracy figure comes from.

Offline evaluation — the exam before release

- A golden dataset of several hundred real tasks is maintained, each with a known-correct expected outcome.
- Cases are drawn from real usage, especially from failures and from tasks that had to be escalated to a human — the hard cases are the valuable ones.
- Every candidate version is replayed against the whole set automatically in the CI pipeline.
- Scoring is layered: exact checks where possible (did the patch compile, was the correct file chosen), and model-graded scoring for open-ended answers.
- Tool-choice accuracy is scored separately from final-answer accuracy, so it is clear whether a regression came from planning or from reasoning.
- If the score falls below the current baseline, the release is blocked automatically. This is the safety net that stops a plausible-looking prompt change from shipping a regression.

Online evaluation — measuring on real traffic

- A new version is released to a small percentage of live traffic first and compared directly against the current one.
- Shadow mode is also used: the new version processes real requests in the background without its output being shown, so its behaviour can be compared risk-free.
- The measures that matter are behavioural: was the suggestion accepted, was the pull request merged, did the user immediately rephrase the same question, how often was a task escalated.
- Guardrail metrics are watched at the same time, so a version that improves accuracy but doubles latency or cost is caught.
- If anything degrades, traffic is rolled back automatically.

Why both are needed

	Offline evaluation	Online evaluation
When it runs	Before release, in CI	After release, on live traffic
What it uses	A fixed set of saved tasks with known answers	Real user requests
Main strength	Repeatable and comparable — the same exam every time	Honest — it measures what actually happens in the real world
Main weakness	The test set can drift away from how people really use the system	Slower, noisier, and a bad version reaches some real users
What it prevents	Shipping a known regression	A regression that the test set never anticipated

Where the 35% came from

Accuracy here means the share of real production tasks that ended in a correct, usable result — measured on live traffic, not in a lab.

The gain was not one change. It was an accumulation of several, each proposed, measured offline, then confirmed online:

Better retrieval, so the model saw the right code and documents before answering.

Rewritten tool descriptions and schemas, which sharply reduced wrong tool choices.

Permission-scoped tool menus, which shrank the choice to only what was actually available to that user.

The critic node, which caught confidently wrong answers before they were shown.

Escalation on low confidence, which converted a wrong answer into a human-handled one.

The measured order matters: without the evaluation system, none of these could have been shown to help rather than merely felt to.

10. How the headline numbers were achieved

Sub-second latency

The measure is time to first useful output, which is what a user experiences as speed. The techniques that produced it:

- **Streaming.** Output is pushed as it is generated rather than held until complete.
- **Model routing.** Small, fast models handle the frequent, simple calls; the large model is reserved for hard reasoning.
- **Semantic caching.** Repeated questions return in milliseconds without touching a model at all.
- **Asynchronous I/O.** The server does other useful work while waiting on models and tools.
- **Parallel tool calls.** Independent lookups run at the same time instead of one after another.
- **Connection pooling and warm clients.** Connections to databases and model providers are kept open, avoiding setup cost on every call.
- **Tight timeouts and circuit breakers.** One slow downstream system cannot drag everyone's latency up.

1,000+ daily users

- **Stateless API servers.** All state lives in Postgres and Redis, so more instances can be added behind a load balancer at any time.
- **Queued long tasks.** Long-running agent work goes to background workers rather than occupying a web connection.
- **Per-user and per-team quotas.** No single caller can consume the shared capacity.
- **Graceful degradation.** If a tool is down, the agent reports it and works around it rather than failing the whole request.
- **Cost controls.** Routing, caching and budget guards keep the per-task cost low enough for the usage to scale.

35% higher production accuracy

Covered in detail in Section 9. In short: better retrieval, better tool descriptions, permission-scoped tool menus, a critic node that re-checks the work, and escalation instead of guessing — each one proposed, measured offline, and confirmed on live traffic before being kept.

11. Technology stack summary

Area	Technology	Why it was chosen
Language	Python	The AI and machine-learning ecosystem lives here, so every relevant library is available.
Web framework	FastAPI	Asynchronous, fast, and validates request data automatically through type hints.
Agent orchestration	LangGraph	Models the agent as a graph with shared state, built-in checkpointing, and interrupts for human approval.
Tool integration	MCP	One standard interface for every tool, so new capabilities are added by configuration rather than code.
Data validation	Pydantic	Enforces the shape of requests, tool arguments and model output at every boundary.
System of record	PostgreSQL	Transactional and durable — correct for approvals, checkpoints and the audit trail.
Cache and counters	Redis	Millisecond reads for rate limits, sessions and the semantic cache.
Semantic search	Vector database	Retrieves the code and documents that are actually relevant to the question.
Observability	OpenTelemetry-style tracing and metrics	One connected timeline per run, which is essential for debugging long agent runs.
Evaluation	Golden dataset in CI, plus A/B and shadow traffic	Catches regressions before release and confirms improvements after it.

12. Design decisions and trade-offs

Every choice below cost something. Being able to explain what it cost, and why it was still worth it, is what shows the decisions were deliberate.

Decision	What it bought	What it cost
Route every tool call through one governance layer	An absolute safety guarantee rather than a best-effort one, and a single place to audit.	A small amount of latency on every call, and one component that must never fail.
Use MCP instead of custom integrations	New tools are added by configuration; each server holds its own credentials.	An extra network hop, and dependence on a young standard.
Approval gates on writes	No unreviewed change ever reaches a real system.	Slower completion for write tasks, and reviewer time. Mitigated by risk scoring, so only genuinely risky calls pause.
Checkpoint after every step	Runs survive crashes and can pause for hours at no cost.	More database writes and more storage.
Route between a small and a large model	Much lower latency and cost across the great majority of calls.	A routing decision that can be wrong, which itself has to be evaluated.
Add a critic node	Confidently wrong answers are caught before the user sees them.	An extra model call on every task, so more latency and more cost.
Deny by default in permissions	No accidental access; new tools are safe until explicitly granted.	Onboarding friction — permissions must be granted deliberately.
Maintain a golden evaluation set	Regressions are caught before release and improvements can be proven.	Ongoing effort to keep the set current as usage changes.

13. Questions you may be asked, with answers

Q1. What actually makes this "agentic" rather than a chatbot?

A chatbot produces text and stops. This system decides on an action, carries it out with a real tool, looks at the result, and then decides again. It runs a loop of plan, act, observe, revise until the goal is met or a limit is reached — and it changes real systems along the way.

Q2. Why LangGraph rather than a simple loop in Python?

Three things a hand-written loop does not give you for free: shared state that every step reads and writes, checkpointing after each step so a run can survive a crash or a twenty-minute pause, and interrupts, which are what make human approval possible mid-run. The graph is also inspectable, so you can see exactly which path a run took.

Q3. Why MCP instead of just writing integrations?

Without a standard, connecting five systems means five bespoke integrations, each with its own authentication, error handling and output format, and every one of them has to be re-taught to the agent. With MCP, every tool describes itself the same way, the agent discovers them at runtime, and adding a capability becomes a deployment rather than a code change. It also keeps credentials inside each server instead of in the agent.

Q4. How do scoped permissions actually work?

A rule names a subject, a tool, a resource and an operation. Before any call runs, the governance layer evaluates the user's identity against those rules. Anything not explicitly permitted is denied. Crucially, scoping is per resource, not just per tool, so read access to one repository does not imply access to all of them.

Q5. What is the difference between an approval gate and human-in-the-loop escalation?

An approval gate is about authority: the agent knows what it wants to do and asks permission. Escalation is about capability: the agent does not believe it should be the one doing this, because confidence is low or it has failed repeatedly, so it hands the whole task to an expert with full context.

Q6. How is sub-second latency possible when the agent makes many model calls?

The measure is time to first useful output, not total completion. Streaming shows progress immediately, routing sends frequent simple calls to a small fast model, caching returns repeat questions in milliseconds, and independent tool calls run in parallel. A long task still takes a while overall, but it never looks frozen.

Q7. How do you know accuracy improved by 35% rather than just feeling better?

Because it was measured twice. Offline, every change was replayed against a fixed golden dataset of real tasks with known-correct answers, so versions could be compared like for like. Online, the new version ran against the old one on a slice of live traffic, and behavioural outcomes were tracked — whether suggestions were accepted, whether pull requests were merged, whether users had to rephrase.

Q8. What happens when the agent gets it wrong anyway?

Several layers catch it. The critic re-checks the result. Low confidence triggers escalation rather than a guess. Any consequential change was approved by a named human. And the audit log plus checkpoints

mean the run can be replayed exactly to work out what went wrong, which usually becomes a new case in the evaluation set.

Q9. What was the hardest part?

Balancing safety against speed. Gating everything makes the platform unusable; gating nothing makes it unsafe. The resolution was risk scoring, so the large majority of calls, which are reads, run instantly, and only genuinely consequential actions pause for a person.

Q10. What would you improve next?

Broader parallel execution of independent steps, learned rather than hand-tuned routing between models, richer automatic scoring so more of the evaluation set can be graded without a human, and a policy simulator that shows the effect of a permission change before it is applied.

The three sentences to remember

1. It is a platform where AI agents do real engineering work — reading code, running builds, opening pull requests — instead of only talking about it.
2. It is safe because every action the agent proposes is stopped and checked: is this user allowed, how risky is it, and does a human need to approve it first.
3. It is measurably better because every change was tested on a fixed golden dataset before release and confirmed on live traffic afterwards, which is how the 35% accuracy improvement was proven rather than assumed.